Objetivos

Unidad 1 – Lenguajes Regulares y Autómatas

OE1.1 Explicar los conceptos fundamentales de alfabetos, cadenas y lenguajes.

OE1.2 Definir formalmente los distintos tipos de autómatas finitos (deterministas, no deterministas, no deterministas con transiciones lambda, transductores), dar ejemplos, identificar sus elementos y conocer sus aplicaciones.

OE1.3 Reconocer la equivalencia entre expresiones regulares y autómatas finitos.

OE1.4 Entender y aplicar, tanto las nociones de lenguajes y expresiones regulares, como de autómatas de estado finito, para el reconocimiento de patrones, procesamiento, validación y extracción de texto usando un lenguaje de programación.

Unidad 2 – Gramáticas y Lenguajes

OE2.1 Definir una gramática independiente del contexto y construir GICs para lenguajes dados.

OE2.2 Explicar el concepto de ambigüedad lingüística y aplicarlo a GIC ambiguas.

OE2.3 Reconocer la importancia de las formas normales para simplificar GICs.

OE2.4 Aplicar los conceptos de gramáticas generativas para el diseño e implementación de DSLs usando un lenguaje de programación.

OE2.5 Reconocer la importancia de los distintos algoritmos de decisión sobre GIC.

Chomsky: the Code Hazard Observation via Modeling of Syntax and KeY-patterns

Modern software repositories frequently contain security weaknesses that arise directly from the way source code and configuration files are written and managed as structured text. Common threats include hardcoded credentials (e.g., passwords, API keys, access tokens), accidental exposure of secrets through logging or print statements, insecure configuration values (such as plaintext database passwords), and inclusion of internal IP addresses or service endpoints that should remain private.

These issues are particularly critical because repositories are often shared across teams or published to public platforms, where unintended disclosure can lead to unauthorized access, data breaches, or infrastructure compromise. **Chomsky,** the application developed in this project, will address these risks by systematically analyzing repository files as formal language objects, detecting insecure textual patterns, identifying policy violations, and classifying the security posture of each file before such vulnerabilities propagate into deployed systems.

Main Goal

The objective of this project is to develop a Code Repository Security Analyzer that processes source code files and configuration files, detects security vulnerabilities and policy violations, and classifies them into appropriate categories. The system will leverage formal language theory, including regular expressions, finite automata (FA), finite state transducers (FST), and context-free grammars (CFGs), to analyze, validate, and transform code automatically.

Project goals:

1) Code Preprocessing and Tokenization (Regular Expressions)
    a) **Objective:** Extract and standardize key elements of code such as API keys, hardcoded passwords, TODO comments, and suspicious URLs.
    b) Formal definition: Design the needed regular expression and provide an explanation of the language it recognizes.
    c) **Implementation:** Use Python's re module to define regular expressions for identifying sensitive patterns.

d) **Example:**

Detect AWS API Key: r"AKIA[0-9A-Z]{16}"
Detect hardcoded password: r'password\s*=\s*".*?"'
Detect IPv4 address: r"\b(?:\d{1,3}\.){3}\d{1,3}\b"

Code snippet triggering detection:

```
password = "admin123"
api_key = "AKIA1234567890ABCDE"
```

2) Security Classification (Finite Automata)
   a) **Objective:** Classify files into categories such as Safe, Needs Review, or Security Violation based on detected patterns.
   b) Formal definition: provide the 5-tuple (Q, Σ, δ, q_0, F) definition, the type of automaton, and the the transition diagram.
   c) **Implementation:** Build DFAs using pyformlang that recognize sequences indicating insecure coding practices.
   d) **Example:**

Conceptual DFA States: Start → FoundPassword → FoundPrint → Violation

Example code triggering violation:

```
password = "admin123"
print(password)
```

Classification Result: Security Violation

3) Automatic Secure Refactoring (Finite State Transducers)
   a) **Objective:** Automatically transform insecure code into safer alternatives.
   b) **Formal definition:** provide the 7-tuple (Q, Σ, δ, q_0, F) definition, the type of FST, and the the transition diagram.
   c) **Implementation:** Use FSTs to replace hardcoded secrets with environment variable references or remove insecure statements.
   d) **Example:**

Input Code:

```
password = "admin123"
```

Transformed Output:

```
import os
password = os.getenv("APP_PASSWORD")
```

Input code:

```
print(api_key)
```

Transformed Output:

```
# Sensitive output removed
```

4) Configuration Validator (Context-Free Grammars using textX)

You must design and formally specify a context-free grammar (CFG) for a secure configuration language used by the Chomsky application. Your grammar must:

   a) Define hierarchical sections (blocks) that group configuration entries.
   b) Allow recursive nesting of sections (at least one recursive production is required).
   c) Enforce matched delimiters (e.g., balanced braces).
   d) Define well-formed key–value assignments.
   e) Distinguish sensitive keys from regular keys and require that sensitive keys use environment variable references (e.g., ${IDENTIFIER}).

- **Objective:** Validate the structure of configuration files (.env, YAML subset) for secure formatting.
- **Formal definition:** You must present the grammar in formal notation (BNF or EBNF), clearly specifying terminals, non-terminals, start symbol, and production rules. Finally, include a brief justification explaining why the language is not regular and why a context-free grammar is required.
- **Implementation:** Define a CFG with textX for accepted secure configuration formats and generate a validated preview.
- **Example:**

(1) Grammar snippet:

(a) Config: entries*=Entry;
Entry: key=ID '=' value=Value;
Value: STRING | NUMBER | EnvReference;
EnvReference: '${' ID '}';
Secure configuration example:
DB_PASSWORD=${SECURE_DB_PASSWORD}
Insecure configuration example:
DB_PASSWORD=admin123

Step-by-Step Program Outline:

The workflow of *Chomsky* is structured to reflect the theoretical progression studied in formal language theory. Each stage of the system corresponds to a different level of abstraction and a different formal model. The process follows four main phases: text processing and detection, classification, transformation, and validation. Figure 1 presents a proposal of the modules and the workflow of the Chomsky application.

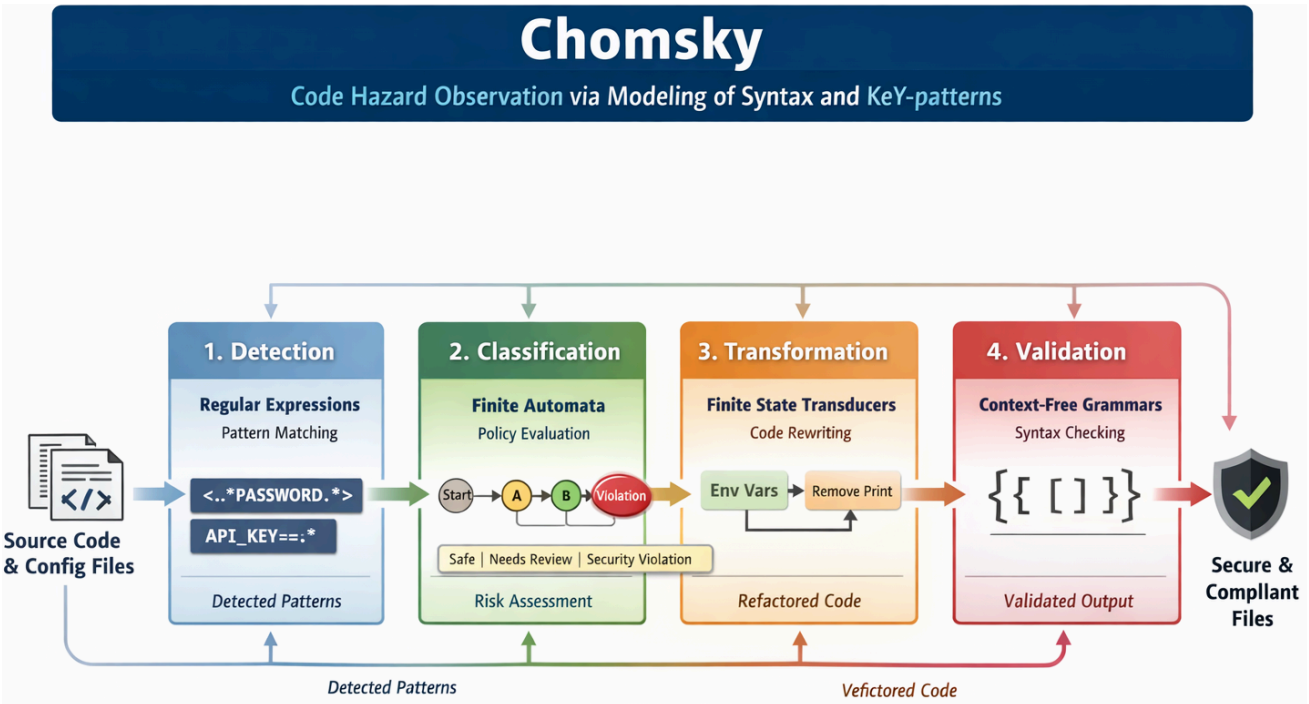

Figure 1: Modules and workflow of the Chomsky app.

1. Detection — Regular Expressions (Lexical Level)
The first stage of the software operates at the lexical level. Source code and configuration files are treated as strings over a finite alphabet. Regular expressions are used to detect predefined textual patterns associated with security risks, such as hardcoded passwords, API keys, suspicious URLs, or exposed IP addresses. At this stage, the system answers the question:

Does this file contain substrings that match known insecure patterns?

The output of this phase is a structured list **of detected tokens or pattern labels** (e.g., HARDCODED_PASSWORD, PRINT_CALL, PLAIN_CONFIG_VALUE). These labels abstract away from raw text and prepare the input for the next formal model.

2. Classification — Finite Automata (Behavioral Level)
The second stage operates over sequences of detected patterns rather than raw characters. The file is now represented as a sequence of abstract symbols generated in the previous step. Finite automata are used to recognize sequences that correspond to policy violations. For example, detecting a password definition followed by printing that password may constitute a stronger violation than either event alone. At this stage, the system answers the question:

Does the sequence of detected events satisfy the conditions of a security violation?

The automaton transitions through states based on the sequence of pattern labels and determines a final classification such as Safe, Needs Review, or Security Violation.

3. Transformation — Finite State Transducers (Rewriting Level)
Once insecure patterns are identified and classified, the system may propose automatic refactorings. Finite state transducers are used to transform insecure constructs into safer alternatives.
For example:

- Replacing hardcoded credentials with environment variable references.
- Removing or commenting out insecure print statements.

At this stage, the system answers the question:

How can insecure textual patterns be systematically rewritten into safer forms?

The transducer maps input strings (or token sequences) to transformed output strings, modeling source-to-source rewriting as a formal language transformation.

4. Validation — Context-Free Grammars (Structural Level)
The final stage focuses on structural correctness. Configuration files often follow a defined syntax that may include nested or structured elements. Context-free grammars are used to validate that configurations conform to secure formatting rules. At this stage, the system answers the question:

Does the structure of the configuration file satisfy the defined secure grammar?

Unlike regular expressions or finite automata, grammars allow the system to reason about hierarchical structure, ensuring that configuration syntax is both well-formed and compliant with security constraints.

CONSTRAINTS IN THE IMPLEMENTATION

- **Integration and User Interaction:**

- Build a CLI or web interface displaying original code, classification result, transformation suggestion, and validation status.

- **Testing and Validation:**
  - Test cases for insecure code patterns, safe code, insecure configurations, and mixed scenarios.
  - Unit tests for regex extraction, DFA classification, FST transformations, and CFG validation.
- **Documentation and User Guide:**
  - Markdown documentation in docs/ directory.
  - README.md explaining setup, dependencies, and usage.
  - Explanation of grammar and automata design.

Literature Review:
Before implementing your program, you should conduct a review of the literature associated with the problem at hand. This is to guide the project, avoid redundancies, make informed decisions, develop a solid theoretical foundation, and thus obtain results of high impact and relevance.

Deliverables

1. Research poster: [Guide]
2. Design:
   a. Design of modules (functions, inputs-outputs)
   b. Formalization
   c. Design of test cases, including scenarios.
3. Python implementation:
   a. Complete and correct implementation of the model, UI, and tests

Project Presentation
Prepare a 10-minute technical presentation (in English) summarizing your project. The presentation must clearly explain the problem addressed, the formal models applied, the system architecture, and the main results obtained. Use your poster to prepare the presentation.

The presentation must be analytical and technical in nature. It should:

- Clearly define the security problem and its relevance in the context of code repositories.
- Explain the methodology, explicitly connecting each system component to its corresponding formal model (regular expressions, finite automata, finite state transducers, context-free grammars).
- Describe the design decisions made (e.g., abstraction level, alphabet definition, grammar structure, classification criteria).
- Present and interpret results, including examples of detection, classification, transformation, and validation.
- Discuss limitations and possible improvements.

This is not a product demonstration or marketing presentation. The focus must be on modeling choices, theoretical justification, implementation strategy, and the rationale of your decisions as engineers to be. Students are expected to demonstrate conceptual understanding of the formal language models they implemented and their practical implications.

Teams
This project can be done in groups of minimum 2 and maximum 3 people.

Departamento de CSI
Computación y Estructuras Discretas III
2026-1 Integrative Task 1

You should create your repository using the following gitHub classroom invitation [link]. Please, fill the following form to register your team, keep the prefix En (where n is a number) in the name of your team [link].

Your repository must have at least 10 commits with a difference of at least 2 hours between each of them. These commits must be meaningful in terms of value (e.g., they cannot be simply deleting a variable or changing a method name). In the repository or project, there should be a directory called docs/ where each of the design documents should be placed, documents in your repository should be written using markdown. Include any necessary clarifications for manipulating or understanding your project in the readme.md file as extra documentation (e.g., IDE used, members' names). The contribution of each member will be evaluated based on their input, and the only way to verify this will be through the commits.

Note: The poster as well as the presentation or demonstration showcasing the functionality and features of the system must be in english.

Deadline for Submission: 12th of April, 2026